Módulo 19

Algoritmos clásicos y patrones para resolver problemas

Temario

1. **Búsqueda lineal**
   - Implementarla manualmente.
   - Devolver índice o `-1`.
   - Mejor caso `O(1)`, peor caso `O(n)`.
   - Cuándo tiene sentido usarla.
2. **Búsqueda binaria**
   - Implementarla nosotros mismos.
   - `inicio`, `fin` y `medio`.
   - Descartar la mitad del problema.
   - `O(log n)`.
   - Requisito: datos ordenados.
   - Errores típicos de límites e índices.
3. **Repaso de algoritmos de ordenamiento**
   - Bubble Sort.
   - Selection Sort.
   - Insertion Sort.
   - Entender cómo funcionan y comparar sus complejidades.
   - No porque vayas a reemplazar `std::sort()`, sino para entrenar razonamiento algorítmico.
4. **Two Pointers**
   - Dos índices moviéndose por una colección.
   - Uno desde cada extremo.
   - Dos avanzando en la misma dirección.
   - Reducir soluciones `O(n²)` a `O(n)` en ciertos problemas.
   - Ejemplos con pares, arrays ordenados y eliminación de duplicados.
5. **Sliding Window**
   - Trabajar con una “ventana” de elementos consecutivos.
   - Sumas de grupos.
   - Máximos/mínimos en subarreglos.
   - Ventanas de tamaño fijo.
   - Ventanas de tamaño variable.
   - Evitar recalcular todo desde cero.
6. **Tablas de frecuencia / Hashing**
   - `unordered_map`.
   - `unordered_set`.

    - Contar ocurrencias.
    - Detectar duplicados.
    - Buscar complementos.
    - Saber si algo ya fue visto.
    - Convertir algunos problemas $O(n^2)$ en $O(n)$ promedio.
7. **Prefix Sum**
    - Crear sumas acumuladas.
    - Consultar rápidamente sumas de rangos.
    - Cambiar consultas repetidas de $O(n)$ a $O(1)$ después de una preparación $O(n)$.
8. **Comparar varias soluciones**
    - Solución sencilla vs solución optimizada.
    - Tiempo vs memoria.
    - Elegir estructuras según el problema.
    - Determinar cuándo optimizar realmente vale la pena.
9. **Problemas combinados**
    - Ya no te diré necesariamente qué patrón utilizar.
    - Tendrás que analizar el problema y decidir si conviene búsqueda binaria, hash, Two Pointers, Sliding Window, etc.
10. **Mini-proyecto / ejercicios finales**

- Resolver varios problemas desde cero.
- Explicar tu razonamiento.
- Analizar complejidad temporal y espacial.
- Empezar a trabajar de una forma mucho más parecida a LeetCode.